Cairo University

Faculty of Computer science and Artificial Intelligence

RL Course

Assignment 1

Dynamic Programming & Model-Based Reinforcement Learning

Overview

This assignment investigates Dynamic Programming (DP) methods within the context of Reinforcement Learning, using the classic Taxi domain as the primary testbed. Students are expected to implement, analyse, and reflect on the following planning algorithms:

- Value Iteration
- Policy Iteration

Upon completing the core implementations, students will extend their work by:

- Constructing a transition model from environment interaction rather than relying on a known model
- Applying their existing design to a more complex environment to assess generalisation

Submission Requirements

- A Jupyter Notebook (.ipynb) containing all implementations, experiments, and analysis with the team names in first cell

Constraints & Guidelines

Constraint	Status
The team must be 5 students and this team not change across the term	
Use of RL libraries (Stable-Baselines, RLlib, etc.)	Not permitted
NumPy and pure Python	Required
Object-Oriented Design (OOP)	Strictly required
Written explanation for every plot	Required
plt.show() outside the Visualiser class	Not permitted
gymnasium (environment instantiation and env.P access only)	Permitted

Section 1 — OOP Implementation

You must implement exactly the five classes described below. Deviating from this structure will result in point deductions.

1.1 TaxiEnvironment

Wraps the Gymnasium Taxi-v3 environment and serves as the sole point of contact with the simulator. This class must expose:

- Total number of states
- Total number of actions
- The transition model P
- Methods: reset(), step(), and render()

Note: *No planning logic of any kind should appear inside this class.*

1.2 ValueIterationAgent

Encapsulates the Value Iteration algorithm. Constructor inputs:

- An environment instance
- Discount factor γ (gamma)
- Convergence threshold θ (theta)

Must implement:

- value_iteration() — executes the full algorithm

Must store the following attributes after execution:

- value_function — NumPy array of size 500
- policy — NumPy array of size 500
- convergence_history — list of max-delta values per iteration

1.3 PolicyIterationAgent

Encapsulates the Policy Iteration algorithm. Must implement:

- policy_evaluation() — iteratively evaluates the current policy
- policy_improvement() — greedily updates the policy

Must store:

- value_function
- policy
- n_policy_changes — total number of policy updates across all iterations

1.4 Planner

Acts as an orchestrator for running and comparing algorithms. Must implement:

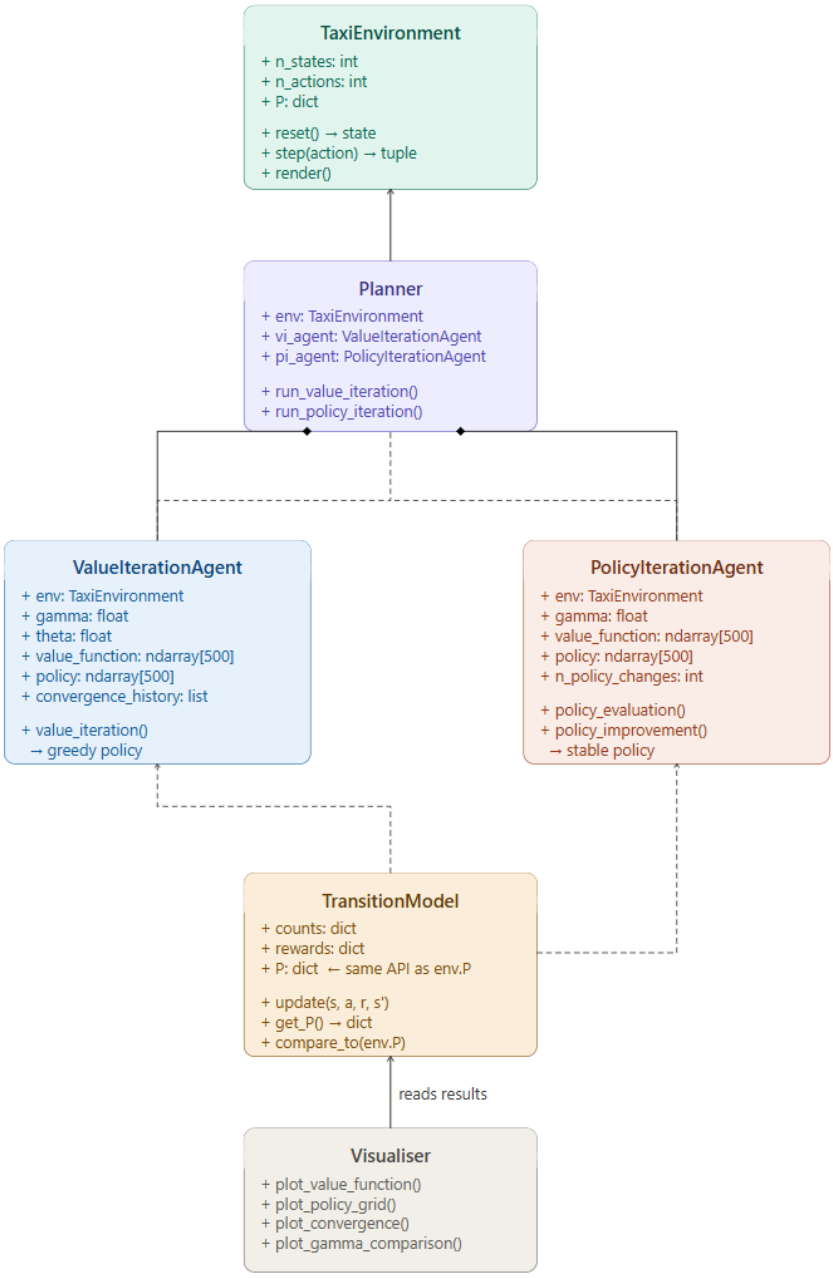
- run_value_iteration()
- run_policy_iteration()
- compare_agents() — runs both algorithms and returns a summary

1.5 Visualiser

All plotting logic must be centralised here. No plot may be generated outside this class. Must implement:

- plot_value_function()
- plot_policy_grid()
- plot_convergence()
- plot_gamma_comparison()

This diagram to help you so you can change of it if it required



Section 2 — Algorithm Details

2.1 Value Iteration

Implement the Bellman optimality update, iterating until the maximum change across all states falls below the threshold θ. The implementation must track and store:

- Maximum delta per iteration

- Total number of iterations to convergence
- The final value function
- The greedy policy derived from the converged value function

2.2 Policy Iteration

Policy Iteration alternates between two phases until the policy stabilises:

Phase 1 — Policy Evaluation

Evaluate the current policy by iteratively applying the Bellman expectation equation until convergence.

Phase 2 — Policy Improvement

Update each state's action greedily with respect to the current value function. Repeat both phases until no policy changes occur in an improvement step.

Section 3 — Experiments

Run both algorithms under three different discount factor settings:

- $\gamma = 0.50$
- $\gamma = 0.90$
- $\gamma = 0.99$

3.1 Required Comparison Table

γ	VI Iterations	PI Iterations	Policies Identical?	Notes
0.50				
0.90				
0.99				

3.2 Required Plots

1. Convergence curves for Value Iteration (one curve per γ value)
2. Bar chart comparing the number of iterations between Value Iteration and Policy Iteration
3. Heatmap of the final value function
4. Policy grid using directional arrows, with pickup (P) and drop-off (D) markers

Note: *Every plot must be accompanied by a written explanation immediately below it in the notebook.*

Section 4 — Conceptual Questions

Answer all of the following questions in full, well-reasoned prose:

5. How does the discount factor γ influence agent behaviour, and what are the trade-offs between low and high values?
6. What is the theoretical relationship between Value Iteration and Policy Iteration?
7. Why is exploration unnecessary when using Dynamic Programming methods?
8. What is the distinction between planning and learning in Reinforcement Learning?
9. How does having access to a model differ from learning purely through interaction?
10. What would happen if an ε-greedy exploration strategy were introduced into a DP agent?
11. Why is a single greedy policy improvement step insufficient to guarantee optimality?
12. Can Dynamic Programming methods such as Value Iteration be applied to complex environments like GTA V? Justify your answer in terms of state space size, model availability, and computational feasibility.

Section 5 — Learning the Transition Model

5.1 TransitionModel Class

Implement a class that maintains a learned approximation of the environment's transition dynamics:

```
class TransitionModel
```

The learned model must:

- Support 500 states and 6 actions (matching the Taxi environment)
- Expose P[s][a], returning a list of (probability, reward, next_state) tuples, consistent with the format used by Gymnasium

5.2 Online Model Learning

Interact with the environment to collect transition experience and update the model incrementally as new (s, a, r, s') tuples are observed.

Note: *Do not derive new update equations. Reuse the approach introduced in Chapter 2 of the course materials.*

5.3 Integration with Planning Agents

Substitute the learned transition model in place of env.P. The ValueIterationAgent and PolicyIterationAgent classes must require no modification to work with the learned model.

5.4 Verification

Compare the learned model against the true environment model across the following dimensions:

- Accuracy of estimated transition probabilities and rewards
- Convergence behaviour of the planning algorithms
- Similarity of resulting policies

Section 6 — Applying Your Design to a New Environment

Environment: MountainCarContinuous-v0

This section challenges you to apply your existing design to an environment with fundamentally different characteristics.

6.1 Key Challenges

- The state space is continuous, not discrete
- No transition model is provided
- Tabular DP methods cannot be applied directly

6.2 Discretisation

You will need to discretise the continuous state space into bins. Consider the following when designing your discretisation:

- How to map continuous position and velocity values to discrete state indices
- How the granularity of discretisation affects solution accuracy and computational cost

6.3 Task

Reuse your existing class structure to:

13. Build a transition model through environment interaction
14. Apply your planning algorithms to the discretised problem
15. Analyse and report on both what works and what fails

6.4 Expected Outcomes

Perfect results are not expected. The evaluation criteria for this section prioritise:

- A clear analysis of where and why the approach succeeds
- An honest identification of its limitations

- A well-reasoned explanation of any failures

Final Objective

The goal is not only to implement algorithms, but to understand when and why they work — and when they fail.